

Witbank Elite Car Wash

INT316D — Web Development Group Project

Semester 1, 2026

Individual Reflection Statements

Witbank Elite Car Wash

INT316D — Web Development Group Project

Semester 1, 2026

Individual Reflection Statements

Student Name: S.R MLWANE

Student ID: 224567642

Role: QA Engineer / Tester

Introduction

Quality assurance was perhaps the least glamorous role in our team, but I would argue it was among the most consequential. As the QA Engineer and Tester for the Witbank Elite Car Wash project, my responsibility was to stand between our development team and our stakeholders as the guardian of correctness, usability, and reliability. This meant scrutinising every feature the Frontend Developer, Backend Developer, and UI/UX Designer produced — not to find fault, but to find the edge cases, the unexpected behaviours, and the gaps between specification and implementation that inevitably emerge in any software project. This reflection documents my testing methodology, the specific defects I identified, the lessons I learned about collaborative quality culture, and the trajectory this experience sets for my career in software quality.

Technical Contribution and Implementation

My technical approach to testing the Witbank Elite system was structured around three layers: functional testing of individual features, integration testing of the end-to-end booking flow, and usability testing of the UI/UX layer. Without automated testing infrastructure (a gap I will address below), all testing was performed manually using a structured test case document I maintained throughout the project.

For the booking flow — the most business-critical path in the application — I developed a matrix of test cases covering: valid booking submission with all required fields, submission with each required field missing, submission with a past datetime, submission with an invalid email format, duplicate booking detection, and the VIP customer trigger (a customer reaching

their third booking). Each test case documented the input, the expected output, the actual output, and the pass/fail status. This disciplined approach meant that when I reported a defect to the Backend Developer, I could provide a precise reproduction case rather than a vague description.

Among the defects I identified and tracked to resolution were: a form submission that succeeded silently when the service type field was left unselected (fixed by adding server-side validation in BookingController), a datetime input that accepted midnight bookings outside of business hours (fixed with a JavaScript min/max constraint), a price display on the booking confirmation page that showed null when the service package was not found (fixed by adding a null-check in the Thymeleaf template), and an admin dashboard that displayed an incorrect total revenue when a booking was cancelled but not excluded from the sum (fixed by filtering on status in the BookingService revenue calculation).

I also performed cross-browser and cross-device testing of the frontend, discovering that the CSS backdrop-filter property produced visual artefacts on Firefox versions below 103. I documented this as a known limitation with a recommended progressive enhancement fallback, which the Frontend Developer subsequently implemented. This experience introduced me to the concept of browser compatibility matrices and the importance of defining a supported browser baseline at the start of a project.

Collaboration and Group Dynamics

The QA role placed me in a unique relational position within the team — I was the only member whose success depended on finding problems in everyone else's work. Managing this dynamic required a deliberate communication style: I framed defect reports as team wins rather than individual failures, using language like 'we found an issue' rather than 'your code is broken'. This approach, which I adapted from reading about blameless post-mortems in engineering culture, created an atmosphere where developers responded to bug reports with curiosity rather than defensiveness.

My closest collaboration was with the Frontend Developer, whose UI features generated the highest density of testable interactions. We established a workflow where I tested each feature on a feature branch before it was merged, rather than discovering defects in the integrated main branch. This shift-left testing approach dramatically reduced the cost of defect resolution — a bug found before merge requires a single developer's attention; a bug found after merge can require coordinated fixes across multiple layers.

I also worked with the Database Manager to validate seed data against the application's business rules, ensuring that the test environment accurately reflected production-like conditions. Having realistic data — including the VIP customer scenario and the low-stock inventory state — meant that my test cases exercised the full branching logic of the application rather than only the happy path.

Lessons Learned and Future Outlook

The most significant lesson I took from this project is that quality is not a phase at the end of development — it is a property woven into every decision from requirements analysis to deployment. Defects that I found in the integration phase of our project could have been prevented earlier: through clearer acceptance criteria at the feature definition stage, through code reviews that enforced validation patterns, or through automated unit tests that would have caught regressions before they reached me.

This insight has motivated me to study automated testing frameworks — specifically JUnit 5 for backend unit tests and Selenium or Playwright for frontend end-to-end tests. Had we invested two days in building a minimal automated test suite for the booking flow, we would have reclaimed far more than two days in manual regression testing over the course of the project. The return on investment for automation is one of the clearest economic arguments in software engineering.

I also gained a profound appreciation for the user perspective. Usability issues — confusing error messages, non-obvious form interactions, missing confirmation feedback — are often invisible to developers who know the system intimately. As a tester, approaching the application as a firsttime user was a skill I had to consciously cultivate, and one I now consider a core professional competency.

Conclusion

The Witbank Elite Car Wash project taught me that quality assurance is simultaneously a technical discipline and a human one. The technical side — test case design, defect tracking, cross-browser validation, regression testing — can be learned from textbooks. The human side — building a culture where defects are welcomed rather than hidden, where developers and testers collaborate rather than compete, where the shared goal is a product that genuinely works for its users — can only be learned by doing. I am grateful for the opportunity to have done it, and I leave this semester with a clear and passionate direction: I want to build systems that work, and I want to be the person on the team who makes sure they do.